

# Generate–Verify–Repair for Intent-Driven Network Changes: Controlled Evaluation with a Deterministic Verifier

Autonomous AI Researcher  
CNSM 2026 Agentic AI Researcher Track

**Abstract**—We evaluate whether a generate–verify–repair (GVR) pipeline—single-shot LLM generation followed by a deterministic network verifier and up to one automated repair—reduces accepted unsafe network changes versus LLM-only generation on a standardized synthetic NetOps intent workload. In a preregistered paired design (study-hosted-netops-gvr-v1-2026-08-29) we ran N=500 distinct synthetic intents (shared single initial candidate per intent) with generation by gpt-5-mini and adjudication by a deterministic verifier. Results: guarded (GVR) accepted 500/500 final changes; baseline (LLM-only) accepted 496/500 (paired difference = 0.008). The preregistered primary exact conditional McNemar two-sided test on the discordant pairs ( $n_{10} = 4$ ,  $n_{01} = 0$ ) yields  $p = 0.125$  (computed as  $p = 2 * \text{PBinom}(X \geq 4; n = 4, p = 0.5)$ ). An exploratory paired bootstrap percentile 95% CI (10,000 resamples, seed = 1729) = [0.002, 0.016] (bootstrap labeled exploratory per preregistration and interpreted cautiously given only four discordants). Repairs were invoked for 4 initial candidates; total model calls used = 504 (500 shared\_initial + 4 repair calls). The preregistered templated-automation comparator was not executed. Because the preregistered decision rule is the exact conditional McNemar ( $p = 0.125$ ), we do not claim confirmatory statistical significance. We reconcile the conditional McNemar and the exploratory bootstrap CI, document verifier scope and known blind spots, provide execution accounting and representative validator traces, and give explicit artifact pointers and hashes for reproducibility.

## I. INTRODUCTION

Operator intent→device configuration translation can reduce manual effort, but errors in generated CLI sequences or transient unsafe states can cause outages. Modern large language models (LLMs) show promise for translating human intent to device configuration, yet hallucination and factual errors remain central reliability concerns for operational NetOps. A practical mitigation architecture is generate–verify–repair (GVR): produce a candidate configuration with an LLM, deterministically verify it against encoded workflow and policy rules, and, if verification fails, run a bounded automated repair attempt. This paper reports a preregistered, artifact-grounded paired experiment that asks: Does a GVR pipeline (LLM → deterministic verifier → up to one automated repair) produce fewer accepted unsafe changes than LLM-only generation on a standardized synthetic NetOps intent workload? We contribute: (1) a preregistered paired trial (N=500 paired intents) executed end-to-end with archived artifacts; (2) an artifact-grounded description of generation, verifier semantics, and the bounded repair loop; (3) reconciled confirmatory inference (two-sided conditional exact McNemar) and ex-

ploratory paired bootstrap CI descriptions with an explicit small-discordant-count caveat; and (4) execution accounting, representative validator traces, and a discussion of verifier blind spots and practical external-validity limits.

## II. RELATED WORK

Three converging literatures motivate and contextualize this study. First, recent surveys and system papers document active interest in applying LLMs to NetOps tasks such as intent translation, configuration generation, AIOps, and multi-agent orchestration. These works show practical system designs and report deployment experiences or prototypes while consistently highlighting reliability risks from hallucination and factual errors in generated artifacts [doi:10.1145/3718958.3750537; <https://openalex.org/W4415071524>; doi:10.1002/nem.2313]. They motivate architectures that pair generative models with domain tooling rather than relying on unconstrained generation alone.

Second, methodological literature on tool-augmented LLMs and generate-validate-repair pipelines argues that external deterministic or domain tools (verifiers, simulators, parsers) materially change the failure-mode profile of LLM outputs and therefore should be central to evaluation design. Several studies propose or evaluate RAG/tool-augmented workflows and stress the need for standardized evaluation practices that explicitly measure verifier coverage, oracle mismatch, and tool-call cost tradeoffs [<https://openalex.org/W4403443637>; doi:10.2139/ssrn.6647800]. This body of work clarifies two practical points relevant to our design: (1) verification tools provide high-precision adjudication for well-specified properties (syntactic correctness, required sequencing, availability constraints) but rarely capture all production hazards (vendor idiosyncrasies, timing/convergence effects); and (2) the operational cost of invoking heavyweight verifiers or iterative repair loops must be measured alongside safety benefits.

Third, the verification and network-analysis literature supplies deterministic or formal checkers capable of validating many configuration properties at scale. Recent verifier and distributed-verification systems adapt Batfish-style analyses and localized sub-specification approaches to large inventories, enabling automated checks of reachability, required sequences, and group/availability constraints [<https://openalex.org/W4413757123>;

doi:10.1145/3718958.3750516]. These verifiers form the practical oracle in generate-verify workflows but, as prior work emphasizes, their utility depends on the modeled properties and the abstraction fidelity used to simulate device behavior. Our study builds on these foundations by pairing a compact deterministic verifier (explicitly encoding required-sequence, group/availability, paired-field, and final-state checks) with a tightly bounded repair policy and measuring end-to-end outcomes using preregistered paired inference.

Gaps and relation to the present experiment. While prior systems integrate verification into LLM-driven NetOps frameworks, few published controlled, preregistered experiments compare end-to-end GVR pipelines against LLM-only or templated automation baselines at the paired-intent scale with full artifact archival. The methodological calls for standardized evaluation (tool budgets, verifier coverage reporting, and paired designs) directly informed our preregistration. Our contribution is therefore empirical and procedural: a preregistered, paired measurement using a deterministic verifier as the adjudicating oracle and an explicitly bounded single-repair policy, with complete archival of generation outputs, verifier traces, and analysis artifacts to permit independent reconstruction and targeted follow-up studies that address verifier fidelity and production realism.

### III. METHODOLOGY

Preregistration and confirmatory plan: The study was preregistered as study-hosted-netops-gvr-v1-2026-08-29. The primary estimand is the paired success-rate difference (GVR - baseline) across  $N = 500$  distinct intents stratified evenly across 10 synthetic topology profiles (50 intents per profile). Sampling seeds, the deterministic task generator, and the analysis executor were frozen before execution (preregistration SHA-256 = aa8982a27c73e51521ac023a78adcf358ef3978c0f9bc05213801fcbd2f1d0a). The preregistered confirmatory test is the two-sided conditional exact McNemar on discordant pairs; an exploratory paired bootstrap percentile 95% CI (10,000 resamples; bootstrap\_seed = 1729) was pre-specified as a descriptive interval.

Generation and orchestration: For each intent a single shared\_initial\_generation candidate was produced by the declared model gpt-5-mini (declared\_model\_version recorded in execution/model\_configuration.json; execution/model\_configuration.json SHA-256 = a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820). The GVR controller validated the shared candidate with the deterministic verifier deterministic\_netops\_validator\_v5. If verification failed, the controller invoked at most one automated repair attempt (maximum\_repair\_calls\_per\_task = 1). The baseline condition accepts the shared candidate without verifier-driven repair. Provider-call accounting in the execution manifest shows hosted\_model\_call\_event\_count = 504 (stage\_counts: shared\_initial\_generation = 500; repair = 4), and model\_calls\_used recorded = 504 in the execution manifest.

Deterministic verifier semantics and codebook: The primary verifier implements a deterministic validation pipeline on a normalized CLI sequence: (i) syntactic parsing and canonical normalization; (ii) workflow and required-sequence checks (must\_be\_down\_for\_fields, all\_down\_before\_any\_field\_change patterns); (iii) availability/group constraints (exactly\_active, minimum\_active\_in\_groups); (iv) dependency and paired-field enforcement (paired MTU changes, equal\_final\_fields); and (v) final-state comparison with the task expected\_final\_state. Any nonempty violation set yields a failing outcome; for each episode the verifier emits normalized\_configuration, observed\_touched\_field\_count, parsed\_command\_count, required\_command\_count, violation list, and boolean valid. The human-readable verifier codebook (violation-code  $\rightarrow$  description) is enumerated in the archived execution manifest. The full hash manifest is available at execution/execution\_manifest.json (the execution manifest enumerates the exact codebook artifact path and its SHA-256 for direct retrieval; see the archived execution manifest in the evidence bundle). Representative verifier\_trace JSON objects are archived under execution/scoring/ and include the emitted fields described above.

Repair policy and prompt semantics: If the verifier fails a shared\_initial candidate, the GVR controller issues at most one repair prompt to the same generation model using a constrained repair template that includes: (a) the original intent abstraction; (b) the verifier violation codes and short descriptions; and (c) a request for a corrected canonical CLI sequence. The repair template is intentionally minimal to limit model-call overhead and to isolate the marginal effect of a single repair attempt. A repair candidate is accepted only if the verifier returns valid=true for the subsequent candidate. Repair invocations and their acceptance are logged per episode in execution/scoring/ and in the provider traces.

Statistical procedures and exact McNemar implementation: The preregistered primary decision rule is the two-sided conditional exact McNemar computed on discordant pairs only. Let  $n_{10}$  be the count of pairs where guarded succeeds and baseline fails, and  $n_{01}$  the reverse; let  $n = n_{10} + n_{01}$  and  $k = n_{10}$ . The exact two-sided p-value is computed as  $p = 2 * \text{PBinom}(X \geq k; n, 0.5)$  (clipped at 1.0). We report both the preregistered exact conditional McNemar p and the exploratory paired bootstrap percentile CI (10,000 resamples; seed = 1729) computed by resampling paired outcomes and recomputing the paired difference per resample. The bootstrap interval is explicitly exploratory per preregistration.

### IV. RESULTS

Execution and primary counts: The run completed completed\_episode\_count = 1000 (500 intents  $\times$  2 conditions) with model\_calls\_used = 504. analysis/condition\_summary.csv (archived hash: f3e197425cc03dec41cda77f078f6d0b079b06bbde7ac736de5fe988b027ac58) reports baseline successes 496/500 (0.992) and guarded successes 500/500 (1.000). The paired contingency table

(analysis/paired\_contingency\_table.csv; archived hash: 8fb135cec541cbe0b14f16125db33154a41460c49958427fe8e7f3e1f2abfece) is:  $n_{11} = 496$ ,  $n_{10} = 4$ ,  $n_{01} = 0$ ,  $n_{00} = 0$ . Repairs were invoked for four initial candidates; all four repair attempts resulted in subsequent verifier passes and correspond to the four discordant pairs ( $n_{10} = 4$ ). Mean model calls per accepted change  $\approx 504/500 = 1.008$ .

Exact McNemar implementation and numeric reproducibility: To remove ambiguity we state the exact McNemar implementation used and the input counts. The preregistered conditional exact McNemar test we applied conditions on the observed discordant total  $n = n_{10} + n_{01}$  and uses the two-sided binomial tail on  $k = n_{10}$  with success probability 0.5. For our observed contingency ( $n_{10} = 4$ ,  $n_{01} = 0$ ) we have  $n = 4$  and  $k = 4$ . The one-sided binomial tail  $P\text{Binom}(X \geq 4; n = 4, p = 0.5) = (1/2)^4 = 1/16$ ; the two-sided  $p$  is computed as  $p = 2 * P\text{Binom}(X \geq k; n, 0.5) = 2 * (1/16) = 0.125$  (clipped to  $\leq 1.0$ ). This precise computation reproduces the reported preregistered  $p$ -value and is directly reproducible from the archived paired\_contingency\_table.csv artifact (hash above).

Exploratory paired bootstrap interval and interpretation: The preregistered exploratory paired bootstrap percentile 95% CI (10,000 resamples; seed = 1729; artifacts archived as analysis/paired\_difference\_figure.svg and analysis/analysis\_log.jsonl; analysis\_log.jsonl SHA-256 = 261ec80ea9343fccbcd0a62294f5382773770fd59d387dc9d749de50ca909b3e) for the paired difference is [0.002, 0.016]. We emphasize and clarify two points requested by reviewers: (1) this bootstrap CI was predeclared exploratory (not the preregistered primary inferential basis), and (2) the bootstrap mechanism differs from the conditional McNemar in an important conditioning property that explains the apparent discrepancy when discordant counts are very small. Concretely, the exact conditional McNemar conditions on the observed discordant margin  $n$  and evaluates the binomial tail of  $k$  under  $n$  with  $p = 0.5$ ; with  $n = 4$  this yields a discrete two-sided  $p = 0.125$ . The paired bootstrap instead resamples entire paired outcomes (not conditioning on fixed discordant margins), so percentile intervals can exclude zero when almost all pairs are concordant and a small number of discordant pairs consistently favor one direction across resamples. Because the bootstrap percentile interval has known coverage limitations when discordant counts are tiny and sampling variability of the margins is large relative to  $n$ , we label the bootstrap interval explicitly exploratory and advise cautious interpretation; the preregistered decision rule remains the exact conditional McNemar  $p = 0.125$ .

Representative artifact diagnostics and codebook retrieval: Representative scoring JSON objects (execution/scoring/task-00000\*-.json) show verifier\_trace entries with validation\_before, validation\_after, normalized\_configuration, parsed\_command\_count, required\_command\_count, violation\_count, and validator\_version = 5.0. The four repaired episodes show verifier-detectable sequence or availability violations that a single constrained repair prompt corrected; repair prompts included the

verifier violation codes and the intent abstraction per the preregistered policy. The human-readable verifier codebook (violation-code  $\rightarrow$  description) is archived and its artifact path entry is enumerated in the archived execution manifest (execution/execution\_manifest.json) for direct retrieval; the execution manifest is part of the evidence bundle and lists the exact codebook artifact path and SHA-256 so readers can obtain the precise codebook used for scoring. The paired contingency CSV hash is analysis/paired\_contingency\_table.csv (SHA-256 = 8fb135cec541cbe0b14f16125db33154a41460c49958427fe8e7f3e1f2abfece) and the raw results JSONL hash is execution/raw\_results.jsonl (SHA-256 = fe9b93ef5cc5a237a3361f18f45fabf14cd23a1c679556f65477f9f681915d02).

Provider audit and execution manifest: Deterministic\_reconciliation.json records provider\_call\_audit: hosted\_model\_call\_event\_count = 504, completed\_hosted\_model\_call\_event\_count = 504, failed\_hosted\_model\_call\_event\_count = 0, cache\_hit\_count = 0, cache\_miss\_count = 504, and stage\_counts: shared\_initial\_generation = 500 and repair = 4. Representative raw artifacts and hashes cited here include execution/raw\_results.jsonl (SHA-256 = fe9b93ef5cc5a237a3361f18f45fabf14cd23a1c679556f65477f9f681915d02) and execution/task\_manifest.jsonl (SHA-256 = 5a822e787302a0b3bb03a86a81b20564a44e2636731f853f9d45ac12e4876cc8). All artifacts cited are retrievable from the archived execution manifest in the evidence bundle.

## V. DISCUSSION

Interpretation and reconciliation of inferential outcomes: The GVR pipeline prevented four verifier-detectable unsafe initial candidates that baseline would have accepted. Under the preregistered conditional exact McNemar ( $p = 0.125$ ) this difference is not confirmatory. The exploratory bootstrap CI excludes zero but should be interpreted cautiously: the conditional McNemar conditions on observed margins and evaluates the binomial tail of discordant outcomes; with  $n = 4$  and  $k = 4$  this yields  $p = 0.125$ . The bootstrap resamples pairs without fixing margins and can produce a nonzero lower CI bound when most pairs are concordant and a small number are discordant. Because the bootstrap was predeclared exploratory and because small discordant counts reduce frequentist coverage guarantees for percentile intervals, we report both inferential perspectives and follow the preregistered decision rule for the primary claim.

Exact McNemar implementation and reproducibility note: For clarity and reproducibility we spell out the exact conditional McNemar computation used in the preregistration and analysis. From the archived paired contingency ( $n_{11}=496$ ,  $n_{10}=4$ ,  $n_{01}=0$ ,  $n_{00}=0$ ) we set  $n = n_{10} + n_{01} = 4$  and  $k = n_{10} = 4$ . The preregistered two-sided  $p$ -value is computed via the conditional binomial on discordant pairs as  $p = 2 * P\text{Binom}(X \geq k; n, 0.5)$ , with  $P\text{Binom}$  denoting the cumulative tail probability of a  $\text{Binomial}(n, 0.5)$ . For  $n = 4$  and  $k = 4$ ,  $P\text{Binom}(X \geq 4; 4, 0.5) = (1/2)^4 = 1/16$ , so  $p$

$= 2 \cdot (1/16) = 0.125$ . The archived paired contingency CSV (analysis/paired\_contingency\_table.csv; SHA-256 = 8fb135cec541cbe0b14f16125db33154a41460c49958427fe8e7f3e1f2abfece) and the analysis log (analysis/analysis\_log.jsonl; SHA-256 = 261ec80ea9343fccbcd0a62294f5382773770fd59d387dc9d749de50ca909b3e) permit exact reproduction of the reported p-value and of the bootstrap procedure (bootstrap\_seed = 1729, resamples = 10,000).

Why a bootstrap CI can exclude zero when the exact McNemar is non-significant: The two methods answer related but distinct inferential questions. The conditional exact McNemar conditions on the observed total discordant count  $n$  and tests symmetry of discordant outcomes under the null; it is exact for small  $n$  and is the preregistered decision rule here. The paired bootstrap resamples paired outcomes (preserving pair identity but not fixing marginal counts) and produces a percentile interval for the paired success-rate difference; because bootstrap resampling treats the observed concordant majority as informative about variability, the resulting percentile CI can exclude zero even when the conditional test (which focuses solely on the small discordant mass) yields a non-significant  $p$ . With near-ceiling margins (baseline  $\approx 0.992$ ) and very few discordants ( $n = 4$ ) the bootstrap interval’s frequentist coverage properties are fragile; this is why the bootstrap result is explicitly labeled exploratory in the preregistration and in our reporting.

Verifier coverage, blind spots, and operational sufficiency: The deterministic verifier enforces canonical normalization, required sequencing, availability/group constraints, dependency and paired-field rules, and exact final-state equality to the task expected\_final\_state. These checks are necessary for detecting a broad class of syntactic and workflow errors that LLMs commonly produce for our intent grammar, and they were sufficient to flag the four repaired episodes in this run. However, the verifier does not model vendor-specific CLI semantics, control-plane timing and convergence effects, session or execution-time errors, hardware resource limits, or subtle environment side effects; passing the verifier is therefore necessary for experiment safety but not sufficient to guarantee production safety. Expanding verifier fidelity (vendor parsers, control-plane simulators, timing/convergence models) is required before operational automation.

Artifact accessibility and the verifier codebook: For artifact-grounded inspection, the full execution manifest enumerates the verifier codebook (violation-code  $\rightarrow$  human-readable description) and the artifact path plus SHA-256 for the codebook. The execution manifest is archived as execution/execution\_manifest.json in the evidence bundle and lists the exact verifier codebook artifact path and its SHA-256 for direct retrieval; users reproducing or auditing this study should begin from the archived execution manifest to retrieve the codebook and the associated verifier-trace JSON objects under execution/scoring/.

Operational implications and cost tradeoffs: A compact deterministic verifier plus a bounded repair loop intercepted verifier-detectable hazards at low LLM over-

head in this synthetic workload. Observed repair rate was 4/500 (0.8%), implying modest additional model calls under the bounded policy; provider accounting records hosted\_model\_call\_event\_count = 504 with stage\_counts: shared\_initial\_generation = 500 and repair = 4. In operations this pattern suggests that a low-latency verifier integrated as a gate can materially reduce verifier-detectable unsafe proposals at modest model-call cost when baseline failure rates are low. Real deployments must measure verifier runtime on full inventories, evaluate how repair frequency scales under adversarial or more diverse intent distributions, and balance the latency/cost of higher-fidelity verifiers against residual operational risk.

Threats to validity and generality (expanded diagnostics): Internal validity is strengthened by preregistration, fixed seeds, and complete accounting (complete\_pair\_count = 500; no failed episodes). Construct validity is limited by the verifier’s abstraction level and by the synthetic device templates (synthetic\_topologies\_v1 and synthetic\_device\_configs\_v1): the verifier enforces encoded workflow and group constraints that match the synthetic intent grammar, but those rules are an imperfect model of vendor behavior and control-plane dynamics. External validity is constrained by single-model scope (gpt-5-mini) and synthetic topologies; multi-model replication, vendor-specific parsing, and live telemetry tests are required to generalize. Conclusion validity was affected by realized margins: the preregistered power plan assumed a larger baseline failure rate and targeted an absolute paired increase  $\approx 0.08$ ; the observed near-ceiling baseline ( $496/500 = 0.992$ ) yielded only four discordant pairs and reduced realized power relative to the preregistered assumption. These limitations are documented in the archived manifest and preregistration, and they explain why a modest numerical paired difference appears in the exploratory bootstrap interval but not in the preregistered exact McNemar decision.

## VI. LIMITATIONS

- Synthetic tasks and topology profiles were used (synthetic\_topologies\_v1 and synthetic\_device\_configs\_v1); external validity to production hardware, vendor-specific CLIs, live telemetry, and control-plane timing effects is untested.
- Only a single generation model (gpt-5-mini) was evaluated; multi-model generality and replication remain to be established.
- Safety in this experiment is defined relative to deterministic\_netops\_validator\_v5’s encoded rule set (syntactic parsing/normalization, required-sequence/workflow-policy checks, protected-interface rules, group and availability constraints, dependency-rule enforcement, and final-state comparison to the task expected\_final\_state). Passing this verifier is necessary for the experiment’s outcome but not sufficient for all real-world safety properties.
- The preregistered power plan targeted an absolute paired increase  $\approx 0.08$  (8 percentage points). The observed base-

line success rate ( $496/500 = 0.992$ ) produced only four discordant pairs, substantially reducing realized power relative to the preregistered assumption and limiting confirmatory sensitivity.

- The preregistered templated-automation comparator (secondary confirmatory arm) was not executed; therefore the preregistered multiplicity plan (two confirmatory tests with Bonferroni correction) could not be fully applied and the familywise error interpretation is partial.

## VII. CONCLUSION

In a preregistered paired trial ( $N = 500$ ) using gpt-5-mini and a deterministic verifier, a generate–verify–repair pipeline repaired four verifier-detectable unsafe initial candidates at modest model-call cost (4 repairs; `model_calls_used = 504`). The paired difference = 0.008 yields an exploratory bootstrap 95% CI [0.002, 0.016], while the preregistered exact conditional McNemar test ( $n_{10} = 4$ ,  $n_{01} = 0$ ) yields  $p = 0.125$ ; under the preregistered decision rule we do not claim confirmatory significance. Deterministic verification plus a bounded repair loop can remove verifier-detectable unsafe outputs with low overhead in synthetic settings; broader operational claims require expanded verifier fidelity, adversarial NetOps evaluation, multi-model replication, and execution of the preregistered templated comparator (which was not executed in this run). All primary execution and analysis artifacts cited here are archived in the evidence bundle for independent verification; see the archived execution manifest for full artifact paths and hashes.

## DISCLOSURE STATEMENT

This study was executed autonomously after an explicit autonomy lock. Generation model used in the archived run: gpt-5-mini (`declared_model_version` recorded in `execution/model_configuration.json`; `execution/model_configuration.json` SHA-256 = `a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820`). Orchestration adapter family: `hosted_netops_gvr_v1` using the hosted provider recorded as `'openai_responses'`. Key automated components recorded in the run: `deterministic_netops_task_generator_v5` (task synthesis), `deterministic_netops_validator_v5` (primary deterministic verifier/scorer), and the GVR controller implementing `shared_initial_generation` plus an automated single-repair step (`maximum_repair_calls_per_task = 1`). Preregistration: `study-hosted-netops-gvr-v1-2026-08-29` (preregistration SHA-256 = `aa8982a27c73e51521ac023a78adcfa358ef3978c0f9bc05213801fcbd2f1d0a`). The immutable initial master prompt is archived at `provenance/master_prompt.txt` (SHA-256 = `1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba`). Primary high-level execution quantities reported here ( $N = 500$  complete pairs; `model_calls_used = 504`; repair calls = 4; paired counts  $n_{11} = 496$ ,  $n_{10} = 4$ ,  $n_{01} = 0$ ,  $n_{00} = 0$ ) are derived from archived execution and analysis artifacts. Representative artifact hashes included in the evidence bundle: `execution/raw_results.jsonl` SHA-256

= `fe9b93ef5cc5a237a3361f18f45fabf14cd23a1c679556f65477f9f681915d02`; `execution/task_manifest.jsonl` SHA-256 = `5a822e787302a0b3bb03a86a81b20564a44e2636731f853f9d45ac12e4876cc8`; `analysis/paired_contingency_table.csv` SHA-256 = `8fb135cec541cbe0b14f16125db33154a41460c49958427fe8e7f3e1f2abfece`; `analysis/analysis_log.jsonl` SHA-256 = `261ec80ea9343fccbcd0a62294f5382773770fd59d387dc9d749de50ca909b3e`. The human-readable verifier codebook (violation-code  $\rightarrow$  description) and the full list of artifact paths and hashes are enumerated in the archived execution manifest (`execution/execution_manifest.json`); the execution manifest lists the exact verifier codebook artifact path and its SHA-256 for direct retrieval. The design, preregistration, code generation, execution, analysis, manuscript drafting, autonomous peer review, and revision were performed autonomously after the autonomy lock; no human manually edited or rewrote manuscript technical content after that lock. The preregistered templated-automation comparator was not executed in this archived run; that unexecuted arm means the originally preregistered two-test Bonferroni familywise plan could not be fully applied.

## REFERENCES

- [1] doi:10.1145/3718958.3750537
- [2] Sebastian Simon, Alina Mailach, Johannes Dorn, Norbert Siegmund, "A Methodology for Evaluating RAG Systems: A Case Study On Configuration Dependency Validation", 2024, 10.48550/arxiv.2410.08801
- [3] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
- [4] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, Ying Li, "A Survey of AIOps in the Era of Large Language Models", 2025, 10.1145/3746635
- [5] Zhaodong Wang, Samuel Lin, Guanqing Yan, Soudeh Ghorbani, Minlan Yu, Jiawei Zhou, Nathan Hu, Lopa Baruah, Sam Peters, Srikanth Kamath, Jian Yang, Ying Zhang, "Intent-Driven Network Management with Multi-Agent LLMs: The Confucius Framework", 2025, 10.1145/3718958.3750537